Diabetes Insight

High Level Design Document

Project Type: University-Level AI and Web Application Project

Technology Stack: Streamlit, Python, scikit-learn, joblib, CSV Storage

Document Style: Formal HLD aligned to the approved project template

Table of Contents

Section	Title
1	Introduction
2	System Environment
3	Behavioral Description
4	Alternate Design Methods Considered
5	Process Model
6	Data Model
7	Prototype
8	Standards
9	Construction Strategy
10	Backup and Recovery Procedures
11	Design Constraints
12	Design Criteria
13	Reliability Requirements
14	Anticipated Changes
15	List of Reusable Components
16	List of Third Party Components
17	List of Time Critical and Periodic Functions
18	Cross References with Software Requirement Specification

1. Introduction

1.1 Purpose

The purpose of this High Level Design document is to define a clear architectural model for the implementation of **Diabetes Insight**, a web-based diabetes risk prediction system built with Streamlit and a pre-trained Random Forest model. This document bridges the gap between the Software Requirements Specification and the actual implementation approach by showing how the major system modules, data flow, user interaction, and supporting components fit together.

The HLD is intended to support university evaluation, technical review, implementation planning, and future enhancement of the application. It provides a design view that is detailed enough for academic assessment while remaining understandable for student developers and project supervisors.

1.2 Scope

The scope of this design covers the complete high-level solution for the Diabetes Insight application, including the Streamlit presentation layer, user login workflow, gender-sensitive form behavior, preprocessing and inference pipeline, result classification logic, CSV-based persistence, and exception handling. The design assumes that the machine learning model has already been trained and stored as a joblib artifact before runtime.

The document does not include low-level code design, algorithm training mathematics, or production-grade deployment architecture. Instead, it focuses on the system-level organization needed for a functional, maintainable, and presentation-ready academic project.

2. System Environment

2.1 System Input

The system accepts two main categories of input:

- **User identification input:** name and gender.
- **Medical prediction input:** pregnancies, glucose, blood pressure, skin thickness, insulin, BMI, diabetes pedigree function, and age.

The pregnancies field is dynamically controlled by the system. It remains editable for female users and disabled or fixed to zero for male users. All input values are collected using Streamlit widgets and then validated before prediction.

2.2 System Output

The system produces both on-screen and file-based output:

- **On-screen output:** validation messages, prediction probability, predicted class, risk level, and interpretation text.
- **File-based output:** CSV records containing login details, medical inputs, prediction results, and timestamps.

3. Behavioral Description

The application behavior is event-driven. User actions in the interface trigger system functions that validate data, invoke the trained model, and present results. The table below summarizes the key events.

Event Name	Response To Event	Function
Application Launch	The system initializes the Streamlit UI, session state, and file dependencies.	Startup control
User Login	The system validates name and gender, then creates a session context.	Login management
Gender Selection	The system enables or disables the pregnancies field according to the selected gender.	Conditional UI control
Prediction Request	The system validates input, prepares the feature vector, and starts inference.	Prediction trigger
Model Inference Completion	The system receives class and probability output from the Random Forest model.	Prediction engine
Risk Mapping	The probability is mapped to Low, Moderate, or High risk bands.	Risk classification
Record Save	The system appends a timestamped entry to a CSV file.	Persistence
System Error	The system displays a user-safe error message and blocks unsafe processing.	Exception handling

4. Alternate Design Methods Considered

4.1 Design Methods Analysis

Three implementation strategies were considered during the design stage. The first was a traditional web stack using a Python backend and relational database. This approach offered better scalability and stronger separation between concerns, but it introduced unnecessary complexity for a university mini-project.

The second was a desktop GUI built with a native Python toolkit. While simple to run locally, it was less suitable for demonstrating a modern AI web workflow. The final option was a Streamlit-based solution with file-backed storage. This option was chosen because it aligns directly with the

project requirements, reduces infrastructure overhead, and supports rapid integration with machine learning components.

4.2 Basis on Which the Design Is Arrived

The chosen design was evaluated using the following criteria:

- simplicity of development and deployment,
- compatibility with Python-based machine learning workflows,
- ease of demonstration during university review,
- maintainability by student developers,
- low cost and minimal dependency on external infrastructure.

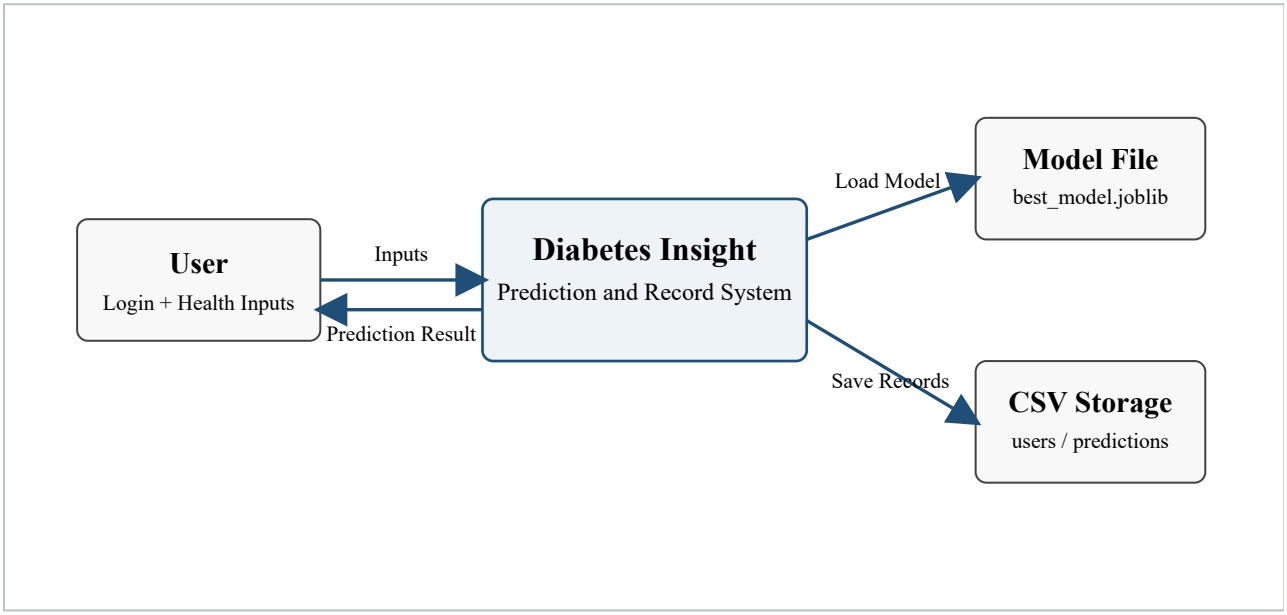
> **Selected approach:** Streamlit + pre-trained Random Forest model + CSV persistence. This option provides the best balance between academic value, ease of explanation, and implementation speed.

5. Process Model

The process model describes how data moves through the system from user interaction to final prediction, display, and storage. The following diagrams present the system context, data flow, decomposition, and object-oriented view at a high level.

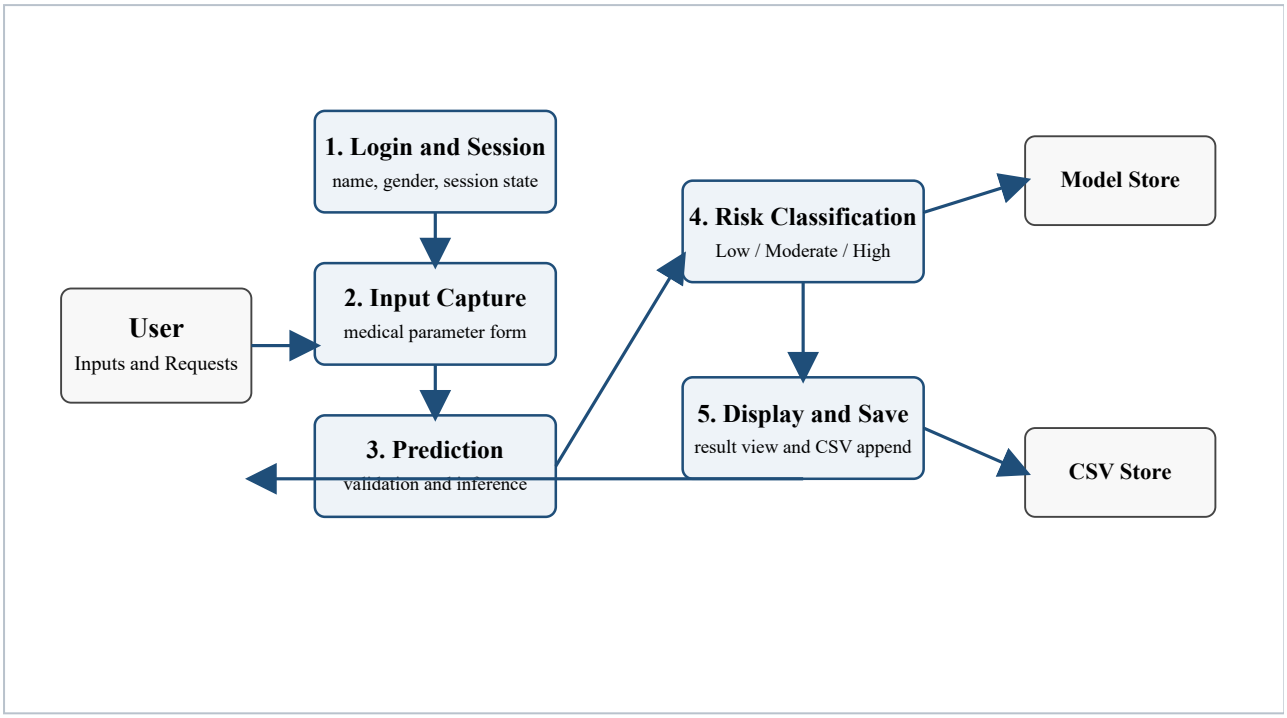
5.1 Context Diagram

Figure 1. System Context Diagram



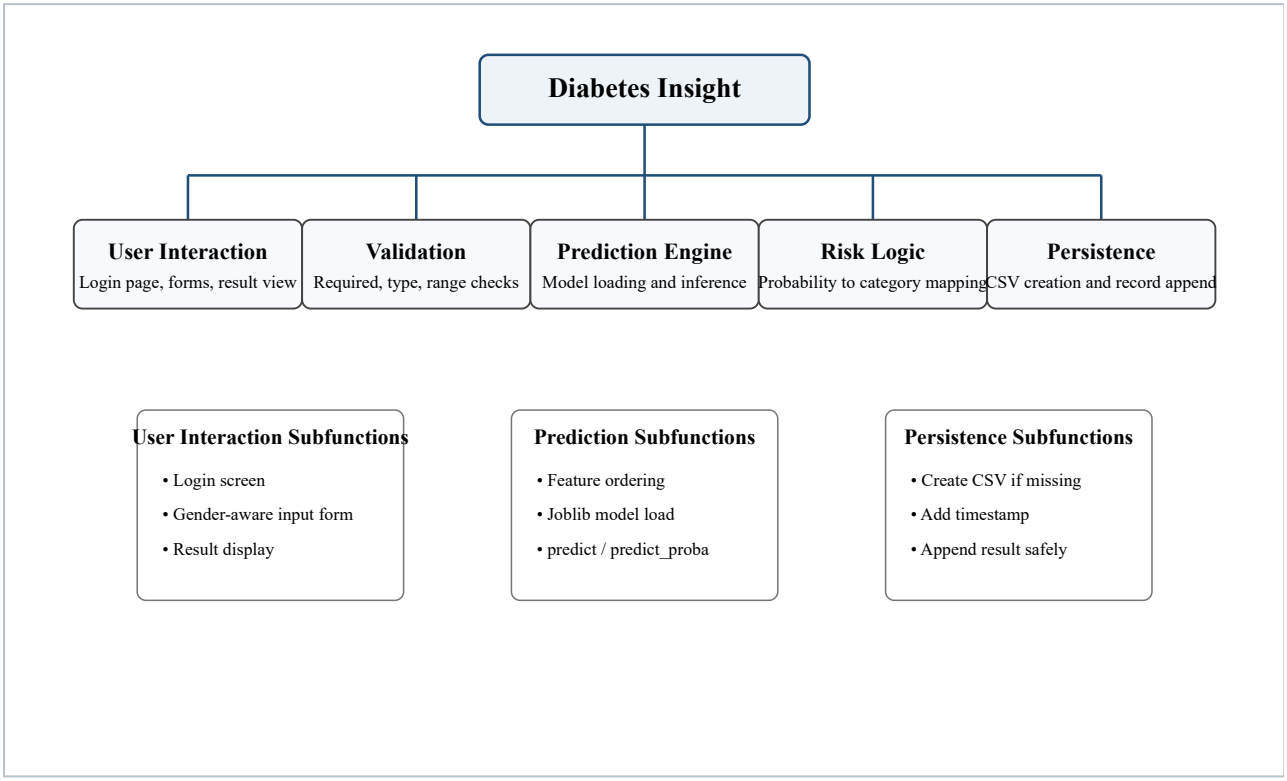
5.2 Data Flow Diagrams

Figure 2. Level-0 Data Flow Diagram



5.3 Functional Decomposition Diagram

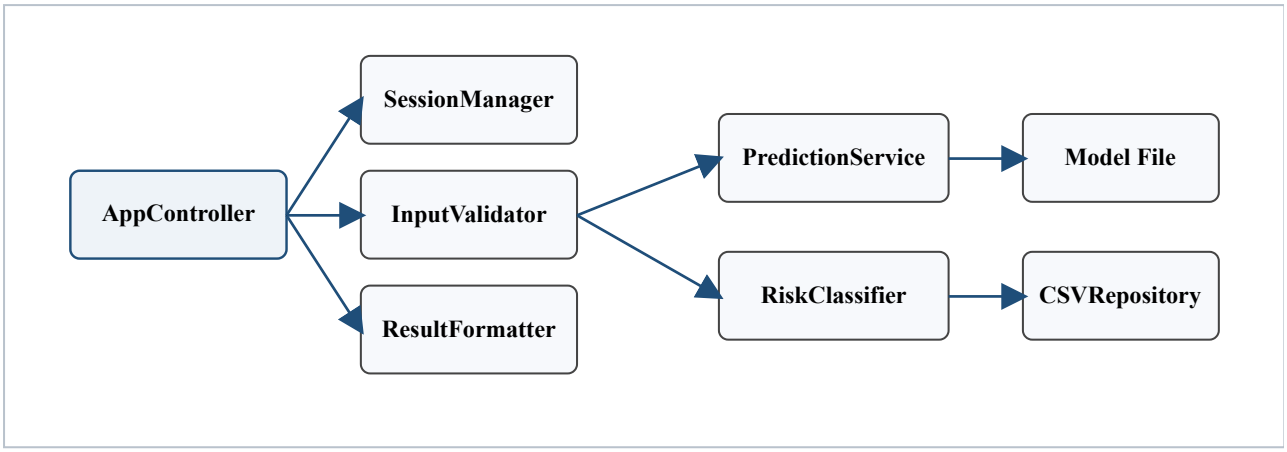
Figure 3. Functional Decomposition



5.3.1 Object Oriented Design

The implementation may remain function-oriented, but the logical design can still be described with object-oriented responsibilities. An *AppController* coordinates user flow, a *SessionManager* holds login context, an *InputValidator* checks the form, a *PredictionService* handles model loading and inference, a *RiskClassifier* maps the probability to risk labels, and a *CSVRepository* manages file persistence. This logical breakdown improves maintainability even if the final code structure is lightweight.

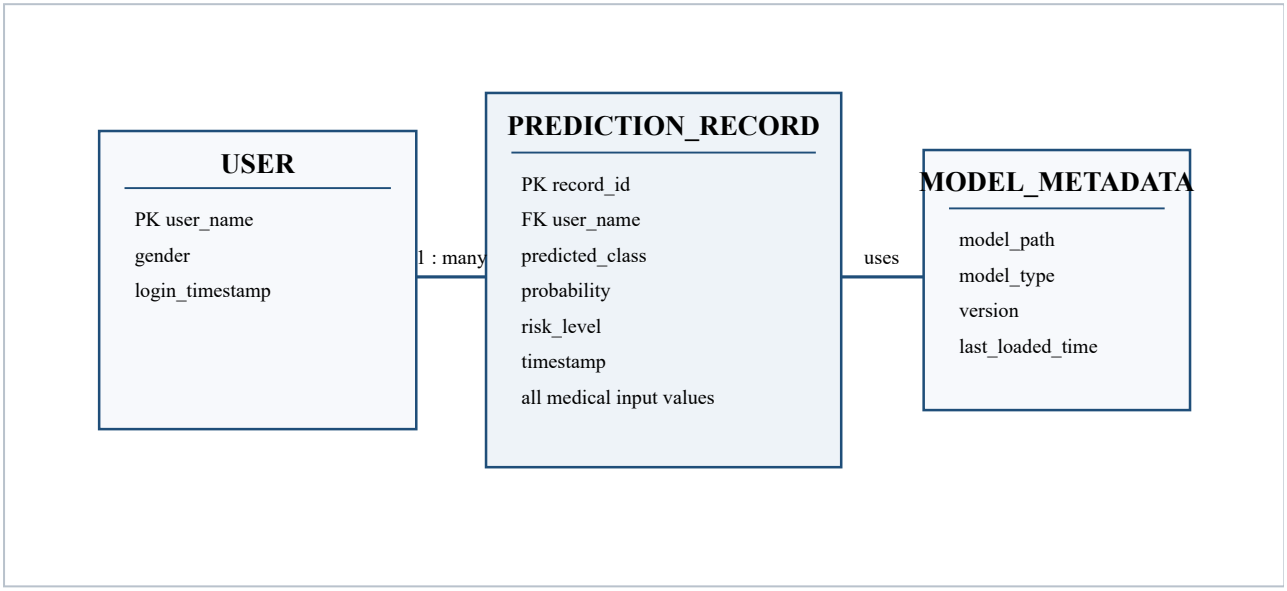
Figure 4. Logical Component Interaction



6. Data Model

6.1 Entity Relationship Diagram

Figure 5. Logical ER Diagram



6.2 Entity, Attribute and Their Relationships

The logical data model contains three principal entities. The **User** entity holds simple login information such as name and gender. The **Prediction Record** entity stores the health input vector, the predicted class, the probability score, the derived risk level, and the timestamp of each prediction. The **Model Metadata** entity represents information about the currently loaded model artifact and is useful for traceability during academic review.

6.3 Data Sizing and Performance

CSV storage is intentionally lightweight. The design assumes small to moderate academic usage, typically from tens to a few thousand rows. At this scale, append operations remain efficient, and the file size is still manageable without indexing or database optimization. Model inference occurs for a single record at a time, so latency remains low on a standard student laptop.

6.4 Processing Specification

1. User login data is captured and stored in session state.
2. Medical input is collected through the web form and validated.
3. Gender-based pregnancies logic is applied before model inference.
4. The feature vector is arranged in the exact order expected by the trained model.

5. The model is loaded using joblib and used to compute the class and probability.
6. The probability is classified as Low, Moderate, or High risk.
7. The final result is displayed and appended to the CSV record store.

6.5 Exception, Signal and Error Handling

Condition	Handling Strategy
Missing name or gender	Display validation error and prevent access to the prediction form.
Invalid numeric input	Reject submission and request corrected values.
Pregnancies entered for male user	Ignore manual entry and force the value to a fixed default.
Missing or corrupted model file	Show a system-safe error message and stop inference.
CSV creation or append failure	Warn the user, log the problem if possible, and keep the UI stable.
Unexpected runtime exception	Handle gracefully and avoid application crash or misleading result display.

7. Prototype

7.1 Scope

The prototype covers the complete user-facing flow required for the project: login, medical data entry, prediction, result display, and record storage. It demonstrates the interaction between the Streamlit UI and the trained Random Forest model in a way that is suitable for university evaluation.

7.2 Objective

The objective of the prototype is to validate usability, confirm successful integration of the machine learning pipeline, and show that the system can guide a user from raw health input to an understandable risk assessment. It also demonstrates the use of a lightweight persistence mechanism without requiring a database.

7.3 Observations

The prototype supports a clean and linear workflow. The conditional pregnancies control improves correctness, model integration remains straightforward due to consistent feature mapping, and the UI is simple enough for non-technical project evaluators to understand during demonstration.

7.4 Conclusion

The prototype is sufficient for academic demonstration and clearly shows how AI prediction, web interface design, and lightweight data persistence can be combined in a practical healthcare-themed student project.

Prototype Screens

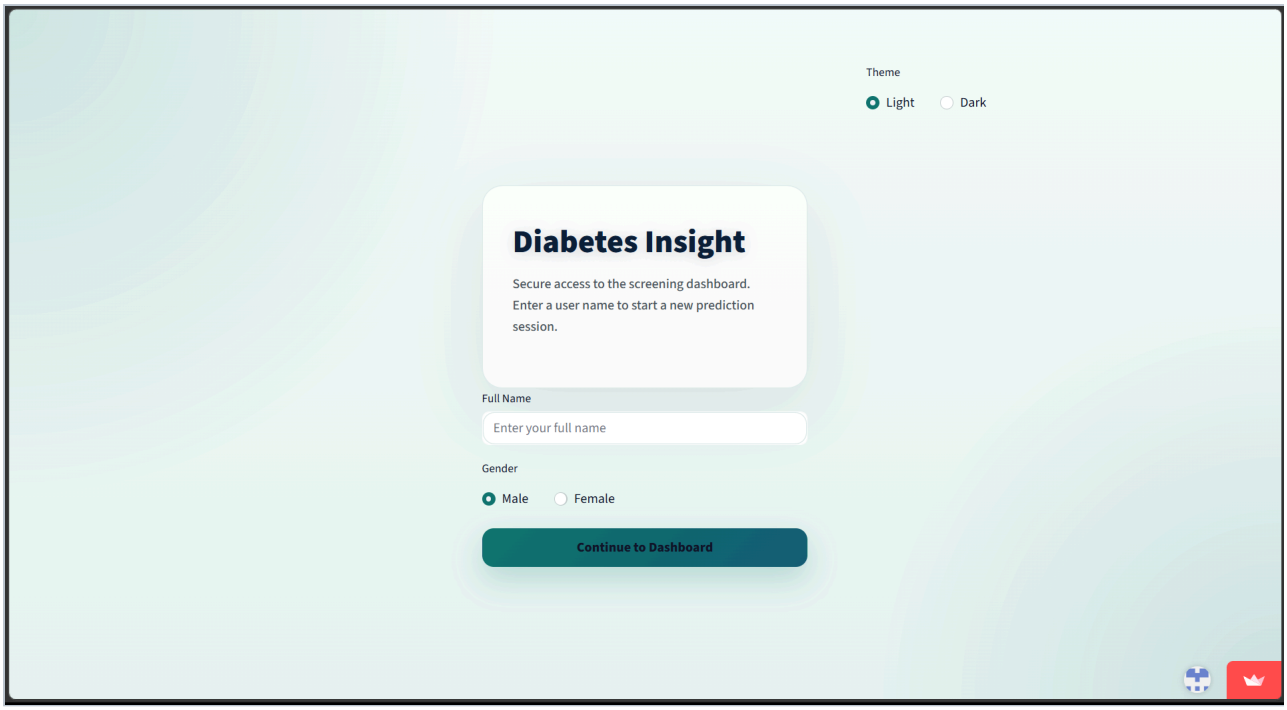


Figure 6. Login page with name and gender capture.

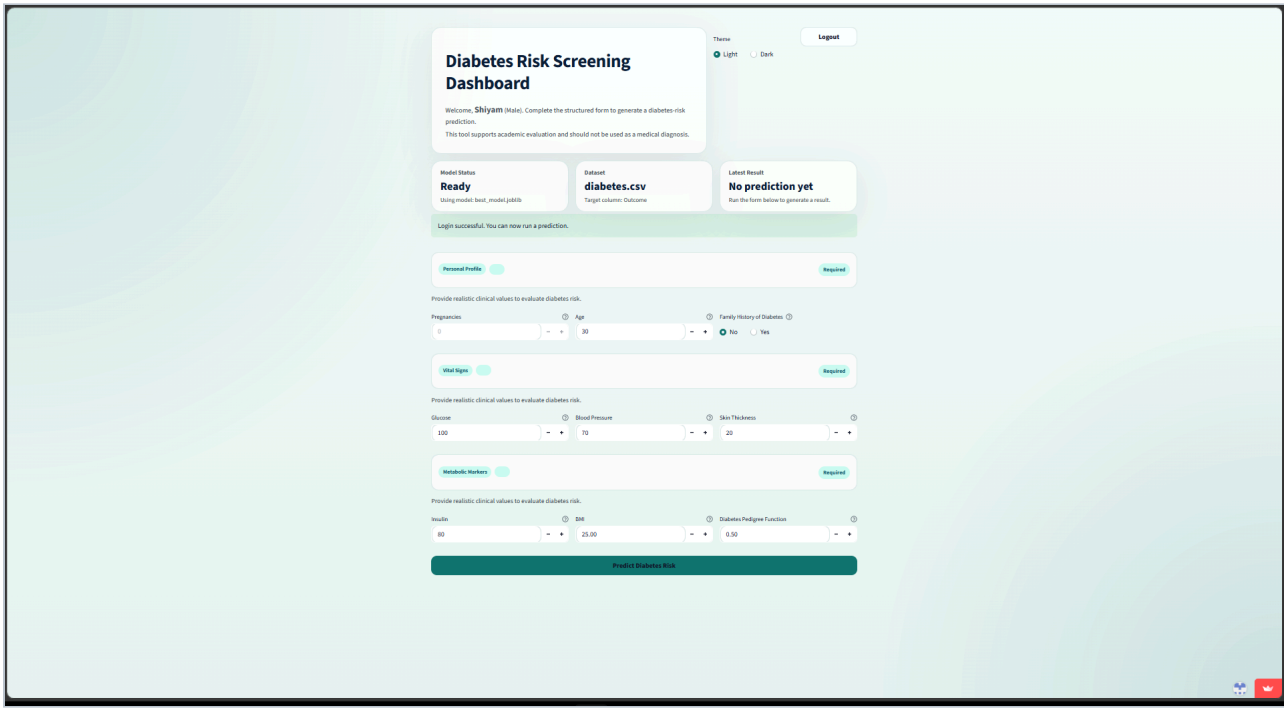


Figure 7. Prediction page with structured medical input controls.

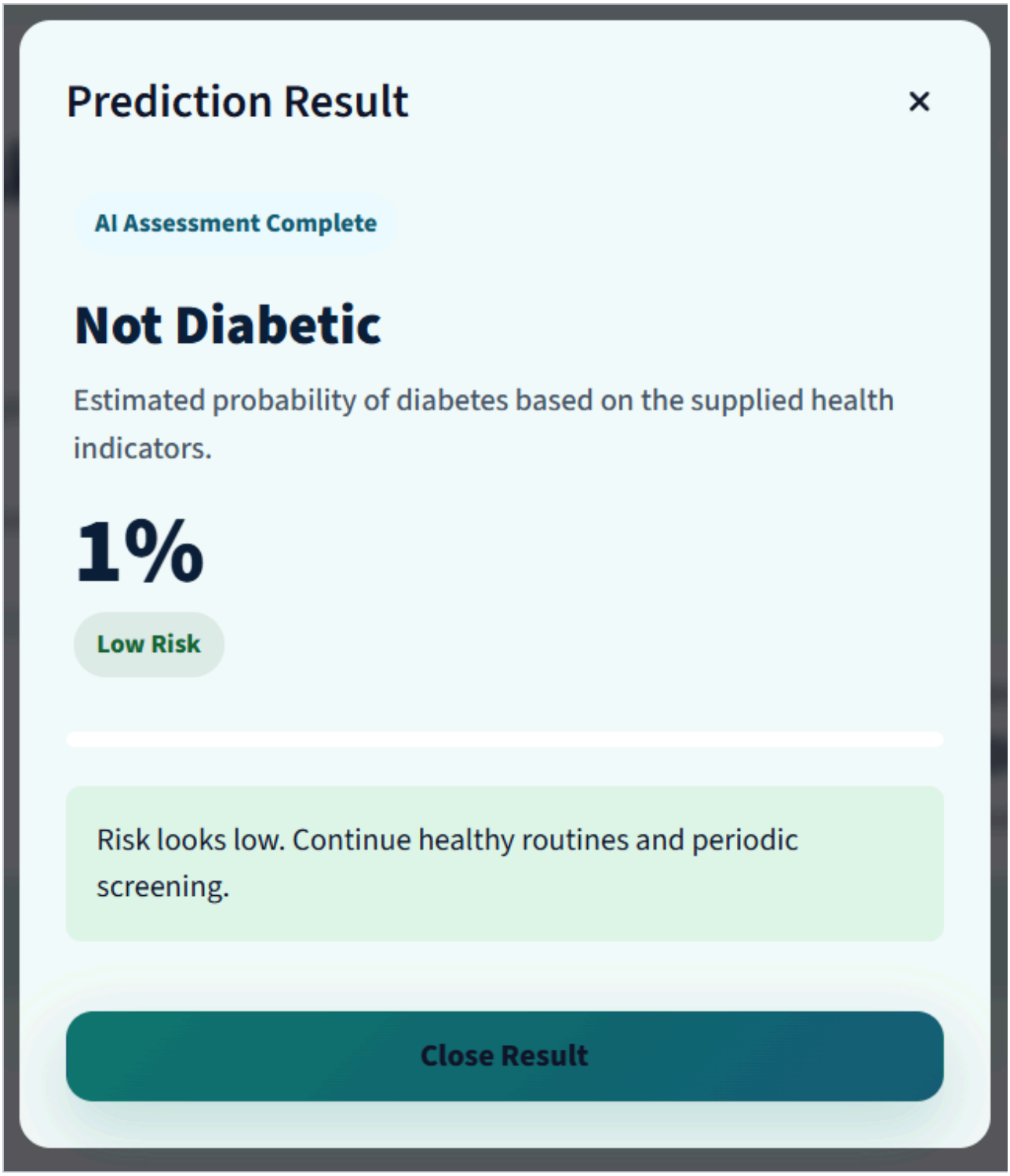


Figure 8. Result page showing prediction output and risk interpretation.

8. Standards

8.1 External Interface Standards

Area	Applied Standard or Convention
User Interface	Streamlit component-based web interface
Programming Language	Python
Model Serialization	joblib
Data Storage	CSV with fixed column headers
Prediction Engine	scikit-learn compatible Random Forest model
Timestamping	Standard date-time text representation for records
Documentation Style	IEEE-inspired academic documentation format

9. Construction Strategy

The construction strategy follows a make-buy-reuse approach. Project-specific business logic such as thresholding, session flow, and conditional pregnancies handling is developed within the project, while established third-party libraries are used for UI rendering, model loading, and tabular data operations.

Sl. No	Module / Function	Buy / Re-use	Verification / Validation	Remarks
1	Streamlit UI framework	Buy	Verify page rendering and form interaction	Presentation layer
2	Random Forest model runtime	Buy / Re-use	Check prediction compatibility with feature schema	Pre-trained model artifact
3	joblib model loading	Buy	Confirm artifact can be loaded without error	Inference dependency
4	CSV persistence utilities	Buy / Re-use	Validate file creation and append logic	Storage support
5	Risk classification logic	Make	Compare output against expected thresholds	Project-specific business logic

6	Gender-aware field handling	Make	UI behavior test for male and female flows	Mandatory project rule

10. Backup and Recovery Procedures

Because the system relies on a model artifact and CSV files rather than a database, backup and recovery are file-centered. The recommended procedure is to maintain a protected backup copy of the joblib model and a periodic backup of CSV records. If the model file becomes unavailable, the application should not attempt prediction until the correct artifact is restored. If CSV files are missing, the system may recreate the file structure, but historical records can only be recovered from backup copies.

11. Design Constraints

- The application must be built with Streamlit.
- The model must be pre-trained and loaded with joblib at runtime.
- Persistent storage must be CSV-based and not database-driven.
- The login process must remain simple and rely on name plus gender.
- The pregnancies field must be disabled for male users.
- The project is intended for educational use and not clinical deployment.

12. Design Criteria

The design is guided by the following criteria:

- **Simplicity:** the system should be easy to explain and demonstrate.
- **Functional completeness:** all required SRS features must be reflected in the design.
- **Modularity:** UI, validation, inference, and storage should remain separable.
- **Maintainability:** future updates such as new thresholds or fields should be straightforward.
- **Responsiveness:** prediction should complete fast enough for interactive use.

13. Reliability Requirements

The high-level design supports reliability through validation-before-inference, deterministic feature ordering, safe model loading, and controlled persistence behavior. The same valid inputs should produce the same outputs as long as the deployed model version remains unchanged. Error conditions must be surfaced as understandable messages rather than silent failures or application crashes.

14. Anticipated Changes

- migration from CSV storage to a relational database,
- support for stronger user authentication,
- multiple model comparison and explainability features,
- additional charts, dashboards, and downloadable reports,
- cloud deployment and multi-user access control.

15. List of Reusable Components

The following project components are reusable across similar academic applications:

- input validation helpers,
- session-state management logic,
- CSV creation and append functions,
- result formatting utilities,
- risk threshold configuration constants,
- single-record prediction wrapper functions.

16. List of Third Party Components

Sl. No	Component	Category	Purpose
1	Streamlit	Third-party framework	Web interface development
2	scikit-learn	Third-party ML library	Random Forest model implementation
3	joblib	Third-party utility	Model serialization and loading
4	pandas	Third-party library	Tabular data and CSV handling
5	NumPy	Third-party library	Numeric data representation where required

17. List of Time Critical and Periodic Functions

Function	Type	Description
User input validation	Time critical	Must complete immediately at submission time.
Model loading	Time sensitive	Should occur quickly or be cached to reduce latency.
Prediction inference	Time critical	Core interactive response must be returned in seconds.
Result display	Time critical	Should appear immediately after inference.
CSV write operation	Time sensitive	Must complete promptly after result generation.
Backup of records	Periodic	Can be performed outside the active user flow.

18. Cross References with Software Requirement Specification

The HLD directly supports the SRS. Login requirements are mapped to the session and UI modules, input requirements are mapped to validation and form-handling logic, prediction requirements are mapped to the inference service, and record-keeping requirements are mapped to the CSV repository component. Quality characteristics such as usability, reliability, maintainability, portability, and efficiency are addressed by the design choices summarized below.

SN	Product Quality Characteristic	HLD Section	Remarks
1	Functionality, suitability, accuracy, compliance, interoperability, security	3, 5, 6, 8, 9, 11	Covered by architecture, process model, and constraints
2	Usability	2, 3, 7, 12	Supported by clear UI flow and prototype design
3	Reliability	6, 10, 13	Handled through validation, error control, and recovery planning
4	Portability	2, 8, 16	Python and Streamlit enable local academic deployment
5	Maintainability	5, 9, 14, 15	Supported by modular structure and reusable components
6	Efficiency	6, 17	Single-record inference and lightweight storage design

> **Conclusion:** The selected design provides a clean, modular, and academically appropriate implementation structure for Diabetes Insight. It is strong enough for university review while remaining practical for student development and future extension.